PRACTITIONER DEEP DIVE

Azure DevOps

Process Types & Work Items

Understanding Basic, Agile, Scrum and CMMI — and how Epics, Features, Stories, Tasks and Bugs fit together to manage real-world work.

For teams & practitioners | Azure Boards · 4 default processes

WHAT WE'LL COVER

Agenda

01 **Core building blocks**
Work items, backlogs, boards and sprints

02 **The four process types**
Basic, Agile, Scrum and CMMI

03 **Choosing a process**
A side-by-side comparison

04 **The work item hierarchy**
Epic → Feature → Story → Task

05 **Each work item type**
What it means and how to use it

06 **A real-world walkthrough**
An online store, end to end

AZURE BOARDS

The core building blocks

Azure Boards is the work-tracking service in Azure DevOps. Four concepts underpin everything else.

01

Work items

The atomic unit of tracked work — an Epic, Story, Task, Bug or Issue. Each has a type, state and unique ID.

02

Backlogs

Prioritized lists of work. Product backlogs hold day-to-day items; portfolio backlogs group them under Features and Epics.

03

Boards

Kanban-style visual views of work items moving through columns, with WIP limits and drag-and-drop.

04

Sprints

Time-boxed iterations (usually 1–4 weeks) that pull items off the backlog into a committed plan.

FOUNDATIONS

What is a “process”?

A process defines the building blocks of work tracking — the work item types available, their fields, states and workflow rules. You pick one when you create a project.

- **Chosen at creation**

  Every project is tied to exactly one process.

- **Locked afterward**

  The process can't be switched once the project exists.

- **Customizable via inheritance**

  Inherited processes let you add fields, states and types without altering the system default.

Process model vs. process

Process model
The framework an organization runs on — Inheritance (modern) or XML (on-premises).

Process
A specific configuration of work item types and rules — Basic, Agile, Scrum or CMMI.

Process template
The XML-based equivalent used by on-premises and hosted-XML models.

OVERVIEW

Four process types, one spectrum

All four ship by default. They differ mainly in their work item types and how much structure they impose — from lightweight to highly formal.

Basic

Epic · Issue · Task

Simplest. Great for small or non-software teams.

Agile

User Story · Task · Bug

Flexible iterative delivery with backlog prioritization.

Scrum

PBI · Task · Bug

Lean, sprint-driven, aligned to the Scrum framework.

CMMI

Requirement · Risk · Review

Most formal — for regulated, high-maturity work.

Lightweight

Formal & structured

PROCESS TYPE 1 OF 4

Basic — the lightest option

WORK ITEM HIERARCHY

- Epic
  - Issue
    - Task

BACKLOG ITEM

Issue — a simple unit of work to do.

BEST FOR

Small teams, newcomers, or non-software projects that want minimal overhead.

HOW BUGS WORK

Tracked as Issues or Tasks; there is no separate Bug type.

BASIC · IN PRACTICE

Real-world examples

Scenario: a small retail team builds an online store using the Basic process — Epic to Issue to Task.

Epic
Business initiative

EXAMPLE

“Launch a new online store for the retail business.”

Issue
A unit of work

EXAMPLE

“Add a product search bar to the storefront.”

Task
Implementation step

EXAMPLE

“Connect the search box to the product catalog API.”

PROCESS TYPE 2 OF 4

Agile — flexible iterative delivery

WORK ITEM HIERARCHY

- Epic
  - Feature
    - User Story
      - Task / Bug

BACKLOG ITEM

User Story — value described from the user's perspective.

BEST FOR

Teams practicing iterative development who prioritize a flowing backlog.

BLOCKERS & BUGS

Bugs sit beside Stories or Tasks; Issues track non-work problems.

AGILE · IN PRACTICE

Real-world examples

Scenario: a food-delivery startup builds its app on the Agile process — Epic to User Story to Task, with Bugs tracked alongside.

Epic Business initiative	EXAMPLE “Launch a same-day food-delivery app.”
User Story Value for a user	EXAMPLE “As a diner, I want to track my order live so I know when it will arrive.”
Task Implementation step	EXAMPLE “Wire the live driver-location feed into the order screen.”
Bug A defect to fix	EXAMPLE “Order status stays on ‘Preparing’ after the kitchen accepts.”

PROCESS TYPE 3 OF 4

Scrum — lean and sprint-driven

WORK ITEM HIERARCHY

- Epic
  - Feature
    - Product Backlog Item
      - Task / Bug

BACKLOG ITEM

Product Backlog Item (PBI) — committed to within a sprint.

BEST FOR

Teams that follow the Scrum framework with sprints and increments.

BLOCKERS & BUGS

Impediments track blockers; Bugs link to PBIs or Tasks.

SCRUM · IN PRACTICE

Real-world examples

Scenario: a healthcare team builds a patient portal on the Scrum process — Epic to PBI to Task, with Bugs linked to their items.

Epic Business initiative	EXAMPLE “Give patients self-service access to their care.”
Product Backlog Item Sprint-sized value	EXAMPLE “Let patients book appointments online without calling in.”
Task Implementation step	EXAMPLE “Build the appointment slot-picker interface.”
Bug A defect to fix	EXAMPLE “Booked slots still show as available to the next patient.”

PROCESS TYPE 4 OF 4

CMMI — formal & structured

WORK ITEM HIERARCHY

- Epic
  - Feature
    - Requirement
      - Task / Bug

BACKLOG ITEM

Requirement — a stack-ranked, formally defined need.

BEST FOR

Regulated or high-maturity teams needing traceability and control.

EXTRA TYPES

Change Request, Risk, Review and Issue support governance.

CMMI · IN PRACTICE

Real-world examples

Scenario: a medical-device team builds infusion-pump software on the CMMI process — Epic to Requirement to Task, with Bugs traced to their parent.

Epic Business initiative	EXAMPLE “Deliver compliant infusion-pump control software.”
Requirement A formal, traceable need	EXAMPLE “The pump must alarm and stop when it detects an occlusion.”
Task Implementation step	EXAMPLE “Implement occlusion-pressure detection in the pump driver.”
Bug A defect to fix	EXAMPLE “Occlusion alarm fires two seconds late at high flow rates.”

SIDE BY SIDE

Comparing the four processes

	Basic	Agile	Scrum	CMMI
Backlog item	Issue	User Story	PBI	Requirement
Portfolio levels	Epic	Epic · Feature	Epic · Feature	Epic · Feature
Bug handling	As Issue / Task	Beside Stories	Beside PBIs	Beside Requirements
Blocker type	Issue	Issue	Impediment	Issue · Risk
Formality	Lowest	Medium	Low–Medium	Highest
Best for	Simple tracking	Iterative teams	Scrum teams	Regulated work

DECISION GUIDE

Which process should you choose?

Want the simplest start?

Basic

A tiny set of types — Epic, Issue, Task — and nothing to learn.

Running classic Agile?

Agile

User Stories plus Bugs give flexible, prioritized iterative delivery.

Practicing Scrum by the book?

Scrum

PBIs, sprints and Impediments map cleanly to Scrum ceremonies.

Need governance & traceability?

CMMI

Requirements, Change Requests, Risks and Reviews enforce control.

HOW WORK BREAKS DOWN

The work item hierarchy

Think of one backlog at multiple zoom levels — each answers a different stakeholder's question.

Epic	Business initiative	*"Are we delivering the initiative?"* **Executives**
Feature	Deliverable capability	*"Which capabilities are done?"* **Product owners**
Story / PBI	Sprint-sized value	*"What are we building this sprint?"* **The team**
Task	Implementation step	*"What are the steps to build it?"* **Developers**

Bugs sit alongside Stories / PBIs or Tasks — defect work tracked in the same flow, planned or unplanned.

WORK ITEM TYPES · PORTFOLIO

Epics & Features

Epic

A large business initiative

- Spans multiple teams and/or releases
- Takes weeks to months — even a quarter
- Represents a business outcome

e.g. “Increase customer engagement”

Feature

A deliverable capability

- A coherent chunk you can demo
- Usually delivered over 1–4 sprints
- Rolls up into an Epic

e.g. “Add shopping cart”

User Stories & Product Backlog Items

THE CLASSIC FORMAT

As a **[persona]**, I want
[an action], so that
[a benefit].

"As a shopper, I want to reset my password so that I can regain access to my account."

Sprint-sized
Small enough to finish inside a single sprint.

Acceptance criteria
Defines exactly what "done" means.

Enough detail to estimate
Lets the team write tasks and test cases.

Focus on the what, not the how
Describes user value, not implementation.

Tasks & Bugs

Task

An implementation step

- Sized to finish within a day
- Links up to a Story, PBI or Requirement
- Lives on the sprint taskboard

Bug

A code defect to fix

- Tracked as a requirement (on the backlog)…
- …or like a task, linked to its parent item
- Teams choose how bugs appear on boards

Blockers & specialty types

Some types never appear on the backlog by default — they track risks, blockers and quality.

Issue / Impediment

Non-work blockers — unclear specs, resource gaps, environment problems. “Impediment” in Scrum; “Issue” in Agile & CMMI.

Test Case

Defines steps and expected results to validate a requirement or story before it ships.

CMMI governance

Change Request, Risk and Review add formal control and traceability in regulated environments.

REAL-WORLD WALKTHROUGH · PART 1

An online store, broken down

Scenario: a retail team is launching online checkout on Azure DevOps using the Agile process.

EPIC Launch a seamless online checkout experience

Feature: Shopping cart

- Story Add & remove items
- Story Save cart for later

Feature: Payments

- Story Pay by card
- Story Apply discount codes

Feature: Order confirmation

- Story Email receipt
- Story Track order status

REAL-WORLD WALKTHROUGH · PART 2

From story to sprint board

The story “Pay by card” is pulled into a sprint, split into Tasks, and moves across the board.

To Do	In Progress	Done
TASK Build payment form UI	TASK Validate card inputs	TASK Design checkout flow
TASK Integrate gateway API	TASK Handle declined cards	TASK Write test cases

IN SUMMARY

Best practices & key takeaways

01 Choose the process up front

It's locked at project creation — match it to how your team really works.

02 Use the hierarchy with intent

Epics for strategy, Features for capabilities, Stories/PBIs for sprint value.

03 Right-size every item

Stories fit a sprint; Tasks fit a day. Avoid epics that are really tasks.

04 Write value, not instructions

Good items say who, what and why — and define what “done” means.

05 Keep blockers visible

Track Issues and Impediments so risks don't hide inside the backlog.

06 Let the tools serve the team

Boards, backlogs and sprints make work visible — not bureaucratic.